

Informe del proyecto

Churn Prediction Service — Servicio MLOps end-to-end

Daniela Casallas · Maximiliano Yañez — MLOps

1. Problema

El proyecto aborda un problema de clasificación binaria: predecir el riesgo de que un cliente de una empresa de telecomunicaciones abandone el servicio (churn). Anticipar este riesgo permite priorizar acciones de retención sobre los clientes con mayor probabilidad de fuga, en vez de aplicar campañas genéricas a toda la base.

El contrato de predicción es simple: dado el perfil de un cliente (datos demográficos, servicios contratados, tipo de contrato y facturación), el servicio entrega la probabilidad de churn y una clasificación binaria según un umbral configurable, expuesto como JSON de entrada y salida a través de una API REST.

2. Datos

Se utilizó el dataset público Telco Customer Churn (IBM/Kaggle), con 7.043 registros de clientes y 21 columnas, incluyendo variables demográficas (género, si es adulto mayor, si tiene pareja o dependientes), variables de servicio (tipo de internet, líneas telefónicas, seguridad en línea, soporte técnico, streaming) y variables de facturación (tipo de contrato, método de pago, cargos mensuales y totales).

La columna TotalCharges llega como texto y contiene 11 registros con espacios en blanco, correspondientes a clientes con antigüedad (tenure) igual a cero; es decir, clientes recién ingresados que aún no acumulan facturación. Para mantener una entrada numérica consistente en el pipeline, estos 11 registros se descartaron durante la limpieza. La columna customerID se excluyó como feature por ser un identificador sin valor predictivo. El dataset no contiene información personal identificable: es un conjunto de ejemplo publicado por IBM con fines educativos.

La variable objetivo (Churn) presenta un desbalance moderado: 5.174 clientes se quedan (73,5%) contra 1.869 que abandonan (26,5%), lo que se tomó en cuenta en el diseño del modelo.

3. Modelo y decisiones de diseño

En vez de asumir a priori que un modelo simple bastaba, se comparó Logistic Regression contra tres alternativas de mayor capacidad — RandomForest, GradientBoosting y XGBoost — sobre el mismo split de datos, con validación cruzada estratificada de 5 folds sobre el conjunto de entrenamiento para medir la estabilidad de cada modelo más allá de una sola partición. La comparación es reproducible: training/compare_models.py corre los cuatro modelos y guarda los resultados en models/model_comparison.json.

Modelo	ROC-AUC (test)	ROC-AUC (CV 5-fold)	Recall	Tiempo entren.
RandomForest	0.8334	0.8472 ± 0.0066	0.7754	0.69 s
LogisticRegression	0.8351	0.8459 ± 0.0054	0.7968	0.05 s
GradientBoosting	0.8328	0.8438 ± 0.0018	0.5134	1.83 s
XGBoost	0.8110	0.8267 ± 0.0088	0.6925	0.14 s

RandomForest obtuvo el ROC-AUC promedio de validación cruzada más alto (0.8472), seguido muy de cerca por Logistic Regression (0.8459). La diferencia de 0.0013 es pequeña en relación con la variabilidad observada entre folds, por lo que ambos modelos mostraron un desempeño predictivo comparable en este experimento.

GradientBoosting y XGBoost, pese a ser más complejos, obtuvieron un ROC-AUC menor; además, Logistic Regression alcanzó el mayor recall en el conjunto de prueba.

Dado ese desempeño comparable, la decisión de producción consideró las variables que sí diferencian a los modelos: Logistic Regression obtuvo el mayor recall en prueba (0.7968), entrenó en 0.05 segundos frente a 0.69 segundos de RandomForest y 1.83 segundos de GradientBoosting, y ofrece mayor interpretabilidad y simplicidad

operacional. La elección se basó, por tanto, en la combinación de desempeño predictivo, recall, costo computacional e interpretabilidad, y no en una preferencia a priori por un modelo simple.

El modelo final combina un preprocesador (StandardScaler para variables numéricas, OneHotEncoder para variables categóricas) con el clasificador, serializados como un único artefacto (model.joblib) junto con metadatos (metadata.json) que registran las features esperadas, la fecha de entrenamiento, la semilla utilizada y las métricas obtenidas. Se usó `class_weight="balanced"` para compensar el desbalance de clases, priorizando no dejar pasar clientes en riesgo real de fuga (mayor recall) a costa de una precisión más moderada — decisión razonable en un caso de negocio de retención, donde el costo de no detectar a alguien que se va suele ser mayor que el de ofrecer una acción de retención a alguien que de todas formas se iba a quedar.

El entrenamiento es reproducible: semilla fija (`random_state=42`) y separación estratificada 80/20 entre entrenamiento y prueba.

4. Resultados

Métricas del modelo elegido (Logistic Regression) sobre el conjunto de prueba (datos no vistos durante el entrenamiento) — ver sección 3 para la comparación contra los otros 3 modelos evaluados:

Métrica	Valor
Accuracy	0.7257
Precision	0.4901
Recall	0.7968
F1-score	0.6069
ROC-AUC	0.8351

El ROC-AUC de 0.835 indica que el modelo separa razonablemente bien a los clientes que se van de los que se quedan. El recall de 0.797 significa que, de cada 10 clientes que efectivamente abandonan el servicio, el modelo identifica correctamente a unos 8 — resultado directo de priorizar recall mediante `class_weight="balanced"`. La contrapartida es una precisión de 0.49: aproximadamente la mitad de los clientes marcados como "en riesgo" no terminan abandonando. Este trade-off es ajustable en tiempo de inferencia mediante el parámetro `threshold` expuesto en la API, sin necesidad de reentrenar el modelo.

5. Arquitectura del servicio

El proyecto separa explícitamente tres etapas: entrenamiento (training/), servicio (app/) y pruebas (tests/), comunicadas únicamente a través de los artefactos versionados en `models/`. La API (FastAPI) carga el modelo una única vez al arrancar, no en cada request, y expone cuatro endpoints: `GET /health` (estado del servicio y del modelo), `GET /model/schema` (features esperadas y métricas del modelo activo), `POST /predict` y `POST /predict/batch` (inferencia individual y por lote).

Todas las entradas se validan con modelos Pydantic estrictos (tipos, rangos y categorías válidas), de forma que una entrada inválida devuelve siempre un 422 con el campo y el motivo del error, nunca un 500 genérico. El servicio se empaqueta en una imagen Docker basada en `python:3.11-slim`, con dependencias fijadas por versión, usuario no-root y `HEALTHCHECK` declarado; docker compose up levanta el sistema completo con un solo comando, sin pasos manuales adicionales.

6. Pruebas y pipeline de CI/CD

La suite de pruebas (13 tests con pytest, sin red ni credenciales) cubre cuatro frentes: el contrato de la API (que `/health`, `/model/schema` y `/predict` respondan con la estructura esperada), la validación de entradas (campos faltantes, tipos incorrectos, categorías inválidas, valores fuera de rango), casos borde de negocio (el umbral de decisión cambia efectivamente la clasificación) y el pipeline de entrenamiento en sí (que los datos se limpian correctamente y que los metadatos guardados son consistentes).

El pipeline de GitHub Actions encadena seis jobs con dependencias explícitas (needs): lint (ruff) → test (pytest) → build (construye la imagen Docker) → smoke (levanta el contenedor recién construido y lo consulta con curl real contra /health y /predict, incluyendo un caso de entrada inválida) → publish (sube la imagen a GitHub Container Registry, solo si se crea un tag v*) → deploy (dispara el despliegue en Render, solo si smoke pasó, y espera hasta confirmar que el servicio público vuelve a responder). Ningún job corre si el anterior falla, y el smoke test consulta el contenedor real construido en el pipeline, no el código fuente directamente — evita el escenario de "funciona en mi máquina" pasando la validación en CI.

7. Despliegue en producción (bonus)

El servicio está desplegado públicamente en Render (plan gratuito) utilizando la misma configuración Docker versionada en el repositorio. La URL pública responde sobre HTTPS: <https://churn-mlops-27t8.onrender.com>; tanto /health como /predict fueron verificados en producción, obteniendo resultados consistentes con el entorno local.

El despliegue está automatizado desde GitHub Actions, no ejecutado a mano: el job deploy llama a un Deploy Hook de Render, almacenado como secreto en GitHub (RENDER_DEPLOY_HOOK_URL), y solo se ejecuta después de que lint, test, build y smoke hayan pasado. Ninguna credencial vive en el código ni en el historial de commits.

Limitación conocida del plan gratuito: el servicio "duerme" tras un período de inactividad, y la primera consulta tras ese período puede demorar entre 20 y 50 segundos mientras el contenedor arranca de nuevo; las consultas siguientes son inmediatas. Esto está declarado en el README para que no se interprete como una caída del servicio.

8. Resultados e interpretación de negocio

Con el modelo en producción (Logistic Regression, threshold=0.5), aproximadamente 8 de cada 10 clientes que efectivamente van a abandonar el servicio son identificados correctamente (recall 0.797), lo que permite dirigir campañas de retención hacia ellos antes de que se vayan. El costo de esa cobertura es que cerca de la mitad de los clientes marcados como "en riesgo" no terminan abandonando (precisión 0.49) — en la práctica, esto significa ofrecer una acción de retención (descuento, contacto proactivo) a más clientes de los estrictamente necesarios, un costo generalmente menor al de perder un cliente sin haberlo intentado retener.

Este trade-off no queda fijo en el modelo: el parámetro threshold expuesto en /predict permite a quien consume la API ajustar el punto de corte según el costo real de cada tipo de error en su contexto de negocio, sin necesidad de reentrenar ni redeployar el servicio.

9. Limitaciones y trabajo futuro

- Ya se compararon 4 modelos (ver sección 3), pero solo con hiperparámetros por defecto o razonables a mano. Con más tiempo, se haría una búsqueda sistemática de hiperparámetros (grid o random search) sobre RandomForest y XGBoost antes de descartar por completo que superen a Logistic Regression con mejor tuning.
- La precisión moderada (0.49) es consecuencia de priorizar recall mediante class_weight="balanced". Como trabajo futuro, el threshold configurable de la API podría calibrarse con costos reales de falsos positivos y falsos negativos del negocio, en lugar de utilizar 0.5 como valor por defecto.
- No hay reentrenamiento automático ni monitoreo de drift en producción: el modelo servido es el que quedó versionado en models/ al momento del build de la imagen.
- El plan gratuito de Render "duerme" el servicio tras un período de inactividad; la primera consulta tras ese período puede demorar entre 20 y 50 segundos. Queda declarado en el README para que no se interprete como una caída del servicio.
- No se implementó autenticación en la API, decisión fuera del alcance de la pauta pero relevante en un despliegue productivo real.

10. Quién hizo qué

El proyecto se desarrolló de manera colaborativa: ambos integrantes participaron en las decisiones de diseño, arquitectura, criterios de validación, estrategia de pruebas y despliegue. Daniela Casallas lideró la implementación técnica inicial del entrenamiento, la API, Docker, el pipeline de GitHub Actions y el despliegue en Render.

Maximiliano Yañez realizó la revisión final de reproducibilidad y calidad en entorno local, ejecutando la suite de tests y linting, validando el contenedor Docker y los endpoints de la API, revisando el flujo de CI/CD y aportando mejoras de documentación y configuración del repositorio mediante un commit y pull request propios.

Ambos integrantes están en condiciones de explicar y defender cualquier parte del código en la exposición oral, dado que las decisiones se discutieron y acordaron en conjunto durante el desarrollo.